AI505

Optimization

Population Based Methods

Marco Chiarandini

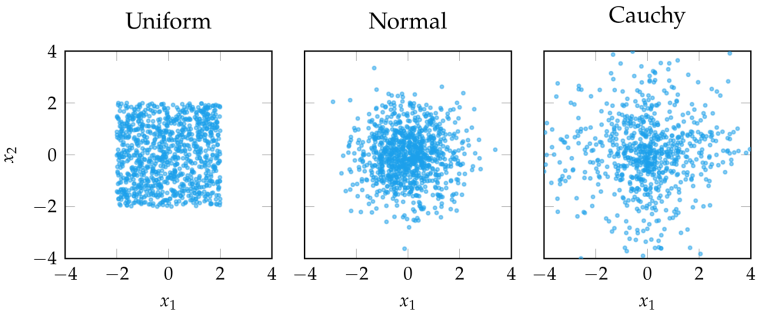
Department of Mathematics & Computer Science
University of Southern Denmark

Population Methods

- Instead of optimizing a single design point, population methods optimize a collection of **individuals**
- A large number of individuals prevents algorithm from being stuck in a local minimum
- Useful information can be shared between individuals
- Stochastic in nature
- Easy to parallelize

Initialization

- Population methods begin with an initial population
- Common initializations are uniform, normal distribution, and Cauchy distribution
- But also space filling designs (later)



Genetic Algorithms

- Inspired by biological evolution where the fittest individuals pass their genetic information to the next generation
- Individuals are interpreted as **chromosomes**
- The fittest individuals are determined by **selection**
- The next generation is formed by selecting the fittest individuals and performing **crossover** and **mutation**

Genetic Algorithms: Chromosomes

- Simplest representation is the binary string chromosome

  but the process of decoding a binary string and producing a design point is not always straightforward.

- Chromosomes are more commonly represented as real-valued chromosomes which are simply real-valued vectors

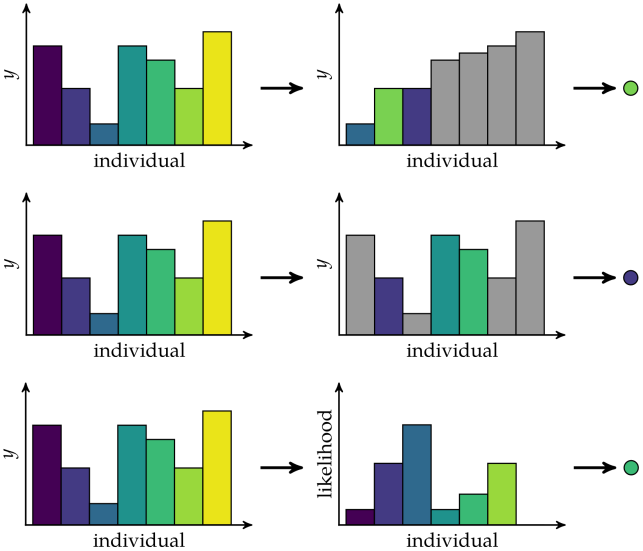
- Typically initialized randomly

Genetic Algorithms: Selection

- Determining which individuals pass their genetic information on to the next generation
  choosing chromosomes to use as parents for the next generation
- **Truncation selection**: truncate the lowest performers
- **Tournament selection**: selects fittest out of k randomly chosen individuals
- **Roulette Wheel selection**: individuals are chosen with probability proportional to their fitness

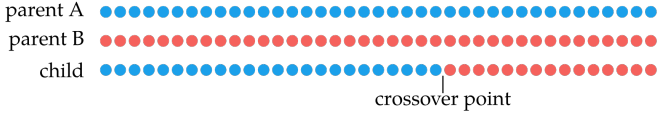
Genetic Algorithms: Selection

Truncation Selection
Tournament Selection
Roulette Wheel Selection

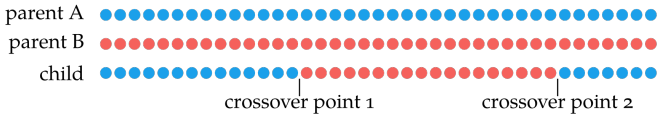


Genetic Algorithms: Crossover

- Combines the chromosomes of the parents to form children
- Single-point crossover: swap occurs after single crossover point



- Two-point crossover: two crossover points



- Uniform crossover: each bit has 50% chance of crossover

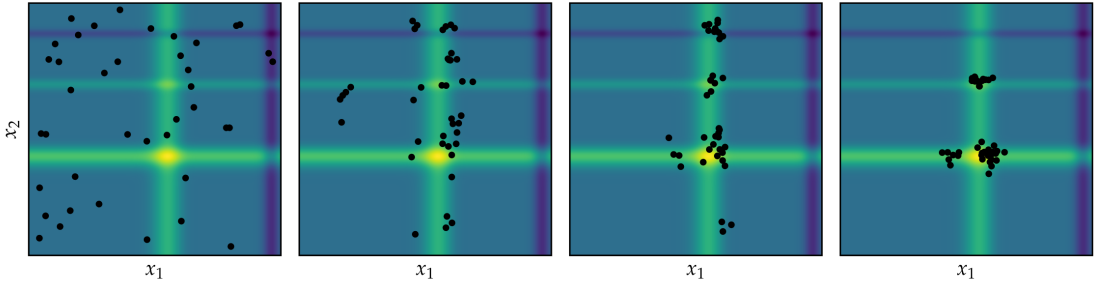
- real values are linearly interpolated between the parents' values $\mathbf{x}_a$ and $\mathbf{x}_b$: $\mathbf{x} \leftarrow (1-\lambda)\mathbf{x}_a + \lambda\mathbf{x}_b$

Genetic Algorithms: Mutation

- Mutation supports exploration of new areas of design space
- Each bit or real-valued element has a probability of being flipped or modified by noise
- The probability of an element mutating is called **mutation rate**

Genetic Algorithms

Genetic algorithm with truncation selection, single point crossover, and Gaussian mutation applied to Michalewicz function



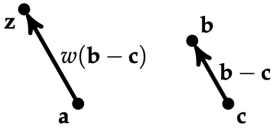
Differential Evolution

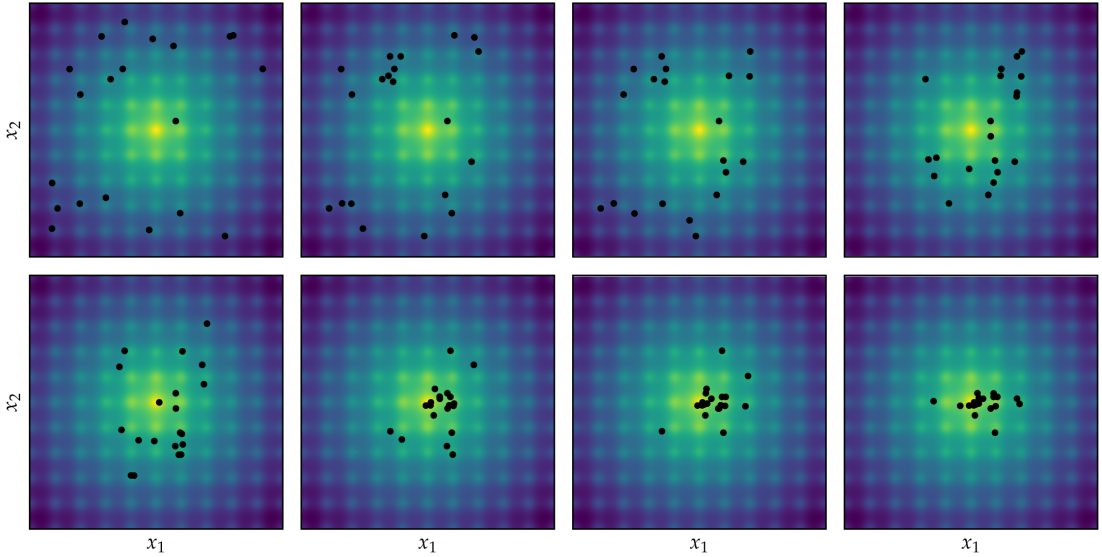
Improves each individual $\mathbf{x}$ by recombining other individuals according to a simple formula

1. Choose three random, distinct individuals $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$
2. Construct interim design $\mathbf{z} = \mathbf{a} + w(\mathbf{b} - \mathbf{c})$
3. Choose a random dimension to optimize in
4. Construct candidate $\mathbf{x}'$ via binary crossover of $\mathbf{x}'$ and $\mathbf{z}$

$$x_i' = \begin{cases} z_i & \text{if } i = j \text{ or with probability } p \\ x_i & \text{otherwise} \end{cases}$$

5. Insert better design between $\mathbf{x}$ and $\mathbf{x}'$ into next generation




x_2
x_2
x_1
x_1
x_1
x_1

Particle Swarm Optimization

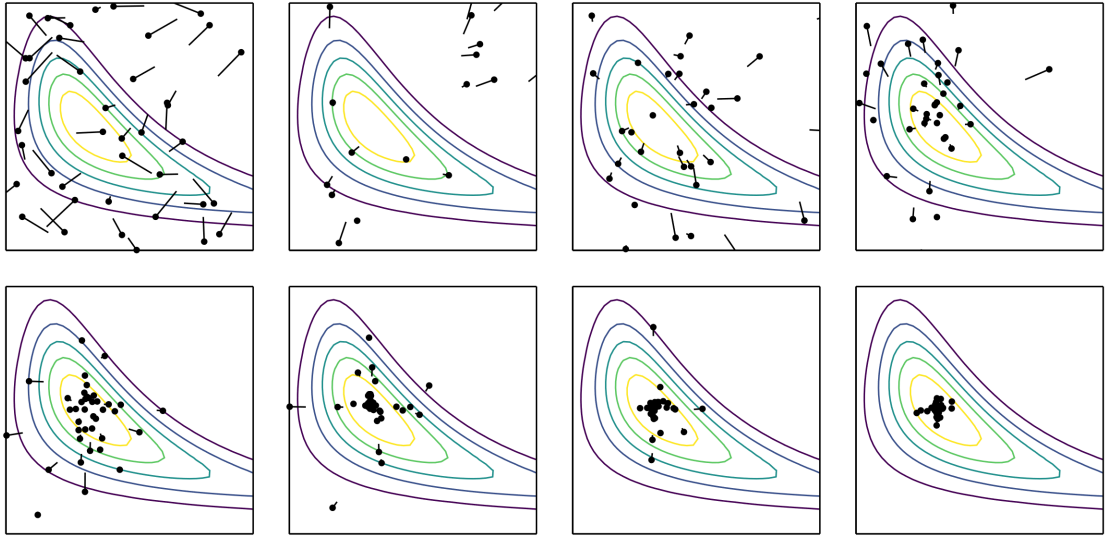
- Each individual, or particle, tracks the following
  - Current position
  - Current velocity
  - Best position seen so far by the particle
  - Best position seen so far by any particle

- At each iteration, these factors produce force and momentum effects to determine the movement of a particle p from the population:

$$x_i^p \leftarrow x_i^p + v_i^p$$
$$v_i^p \leftarrow w v_i^p + c_1 r_1 \left(x_i^{p,best} - x_i^p \right) + c_2 r_2 \left(x_i^{best} - x_i^p \right)$$

$\mathbf{x}_{p,best}$ is the best location among those seen by the particle; $\mathbf{x}_{best}$ is the best location found so far over all particles; w, c_1, and c_2 are parameters; and r_1 and r_2 are random numbers drawn from $U(0,1)$

Particle Swarm Optimization



The Evolutionary Computation Bestiary
https://fcampelo.github.io/EC-Bestiary/

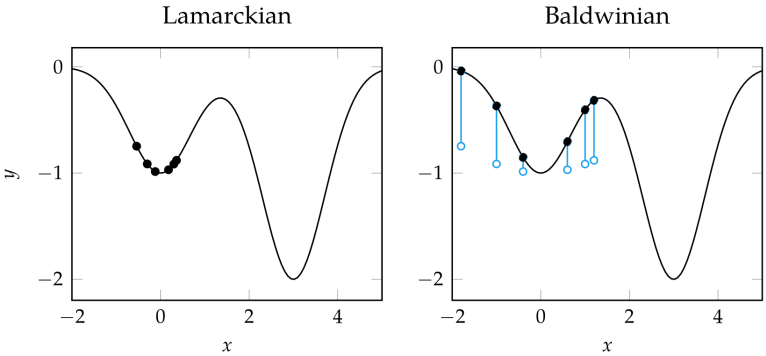
Hybrid Methods

- So far, we have discussed Darwinian Evolution: random variation + natural selection
  Some giraffes are born with longer necks. They access more food. They reproduce more. Long-neck genes spread.
- Generally, these methods are good at finding the best regions in design space, but do not perform as well as descent methods near the minimizer
- Hybrid methods try to leverage the strength of both methods
- Two hybrid approaches
  - Lamarckian evolution: traits that an organism acquires during its lifetime are directly inherited by offspring.
    Giraffes stretch their necks to reach higher leaves. Their necks become longer during life. Their offspring inherit the longer neck.
  - Baldwinian evolution: Learning does not directly change genes, but learning influences which genes are selected. Learning guides evolution indirectly.
    Giraffes stretch their necks to reach higher leaves. Their necks become longer during life. They are more likely to survive and reproduce. Their offspring inherit the genes that make it easier to stretch their necks.

Hybrid Methods

- Lamarckian evolution
  Performs regular descent method update on each individual

- Baldwinian evolution
  Uses value of descent method update to augment the objective value of each design point



Summary

- Population methods use a collection of individuals in the design space to guide progression toward an optimum
- Genetic algorithms leverage selection, crossover, and mutations to produce better subsequent generations
- Differential evolution, particle swarm optimization, (the firefly algorithm, and cuckoo search) include rules and mechanisms for attracting design points to the best individuals in the population while maintaining suitable state space exploration
- Population methods can be extended with local search approaches to improve convergence